

Where Do the Experts Go?

A Validated Speed-of-Light Model for Mixture-of-Experts Decode on Memory-Constrained Consumer Hardware

Harshith Kantamneni
Independent Researcher

Abstract

Frontier-scale Mixture-of-Experts (MoE) models run on desktops only by keeping most experts in host memory. More than thirty systems now propose which experts to cache on the GPU, how to prefetch them, and where to execute the rest. What is missing is a ceiling that all of them can be measured against. We contribute three things. **(1)** A way to record *exact* decode-time routing traces without a GPU. A causal model routes each token according to its prefix alone, so teacher-forced prefill, streamed one decoder layer at a time with tensors read over HTTP, reproduces the routing of KV-cache decode exactly. We verify this against reference implementations of four architectures. With it we trace OLMoE, Qwen3-30B-A3B and gpt-oss-20b over four domains on a 2-vCPU, 7 GB virtual machine. **(2)** A six-parameter bytes-over-bandwidth model of batch-1 decode. It predicts 52 published measurements from 9 independent sources (0.58–128 tok/s; 8 models; CPU-only, static CPU offload and PCIe demand fetch) with 16% median error when whole sources are held out. **(3)** A speed-of-light bound for *any* expert placement policy with a per-layer GPU budget, whether demand-driven or prefetching. It combines Belady’s MIN-with-bypass, a load-accounting argument and Jensen’s inequality. We also give a closed-form reuse threshold r^* that says when copying an expert to the GPU can pay off. The traces show strong short-range reuse: 46% of Qwen3’s selected experts were also selected by the previous token, $7.4\times$ the uniform rate. Long-range popularity, however, is weak and model-dependent across domains; for Qwen3, pinning the experts that were popular on other domains gets a lower hit rate than a static layer split of the same size. Under the validated model, running cache misses on the CPU is never slower than fetching them over PCIe 4 in any configuration we examine. On an 8 GB GPU, a per-layer LRU cache with CPU-executed misses is $1.28\text{--}1.42\times$ faster than llama.cpp’s static layer split and reaches 86–91% of the bound. On faster GPUs holding a quarter of the experts, the static split reaches only 16–28% of the bound and LRU 32–65%, which leaves substantial but bounded headroom for prefetching. Code, traces and the annotated measurement set are released.

1 Introduction

A 120B-parameter MoE with 4-bit experts needs 61 GB for its experts alone, while a consumer GPU has 8–32 GB. Running such models locally means splitting the experts between GPU memory and host DRAM. Decode speed is then set largely by where each routed expert’s weights live when the router selects it.

The literature has responded with many mechanisms:

- demand fetching into an LRU cache [4];
- activation-aware caches and prefetchers [21, 16, 9, 26, 19];
- running misses on the CPU [11, 12, 18, 8, 25];
- mixed-precision or substitute experts [17, 24];
- and, in practice, llama.cpp’s static assignment of whole layers’ experts to the CPU [5].

Recent trace studies disagree on basic questions, such as whether a learned predictor beats LRU [23, 15, 19]. One of them notes explicitly that fewer misses need not mean higher throughput [23].

We argue that what is missing is a *bound*. For a given model, platform and GPU memory budget, how fast can batch-1 decode possibly be, and how close do the strategies people run actually get? A bound turns policy comparisons into fractions of the attainable, as the roofline model [20] does for kernels. Building one takes three things: routing traces of real models, a cost model that is *validated* rather than assumed, and a proof that no policy in the class can beat it. This paper provides all three, using only public data and a CPU-only virtual machine.

Contributions.

- **Exact traces without a GPU** (Section 3). Teacher-forced prefill, streamed layer by layer, matches KV-cache decode routing exactly. Each weight byte is read once per corpus instead of once per token, and never touches the disk.
- **A validated decode model** (Section 4). Six fitted parameters; every other input comes from datasheets and model configurations. The held-out median error is 16% on 52 published points. An ablation shows which terms matter.

- **A speed-of-light bound and r^*** (Section 5). A lower bound on time per token for every per-layer placement policy, including prefetchers, with a regression test against an oracle prefetcher.
- **Findings** (Section 6). Strong short-range locality, weak and model-dependent cross-domain popularity, a quantified advantage for CPU-executed misses on PCIe 4, and the remaining headroom for predictors.

2 Background and Related Work

MoE decode moves data. At batch 1, a routed expert is a pair of matrix–vector products. Its cost is the size of its weights divided by the bandwidth of the memory they are read from. The data-movement view of transformers [10] and sparsity as an efficiency lever [6] frame MoE placement naturally: which bytes cross which link, and how often.

Offloading systems. Demand-paging systems keep a per-layer GPU expert cache and copy misses over PCIe [4, 21, 16, 17]. Prefetchers predict the next layer’s experts [9, 26, 19]. CPU-execution systems run non-resident experts on the host [11, 12, 18, 8, 25]; llama.cpp exposes this as `--n-cpu-moe` and tensor overrides [5]. MoE-Lightning [3] and MoE-Lens [22] model *batched throughput*. We target single-user latency.

Caching theory and methodology. Belady’s MIN [1] minimises misses under demand paging. The interplay of prefetching and caching has been studied for file systems [2]. Recent expert-cache trace studies report miss rates [23] and kernel-managed tiering on GH200 [15]. A related study predicts llama.cpp throughput from GGUF metadata [14]. We follow Hoefer and Belli’s benchmarking guidelines [7]: medians with intervals, explicit fitted parameters, and held-out sources.

3 Exact Routing Traces Without a GPU

Observation. In a causal decoder, the hidden state of token t at every layer is a function of tokens $1..t$ only. The experts that decode selects for token t are therefore the experts that teacher-forced prefill selects at position t , up to floating-point ties.

Layer streaming. Prefill lets us invert the usual loop order. We load *one* decoder layer, push every corpus sequence through it, record the router’s top- k indices, free the layer, and move on. Weights are read once per corpus. The corpus’s activations stay in memory (~ 200 MB for 25k tokens), and peak memory is one layer. Tensors are read straight from the model hub with HTTP range requests, so no checkpoint shard is ever stored. This matters because

gpt-oss-120b’s shards interleave layers and would otherwise need 35 GB of disk. Two further details: the MoE sub-block is token-wise, so after per-sequence attention it runs once over the concatenated corpus and touches each expert once per layer; and gpt-oss experts stay MXFP4-packed and are dequantised one at a time, since a 120B layer would need 13 GB in fp32.

Exactness. Tests compare our per-layer selections with the reference `transformers` models on tiny random checkpoints of four architectures: Qwen3-MoE, OLMoE, DeepSeek-V2, and gpt-oss (both MXFP4-packed and bf16, with sequences longer than gpt-oss’s 128-token sliding window). The references run both as prefill and as genuine token-by-token decode with a KV cache. All selections match exactly. For the real models, teacher-forced perplexity checks that the weights loaded correctly (Table 2). All traces are computed in fp32; Section 8 quantifies how bf16 compute changes routing.

Corpus. We use two-turn conversations from four public datasets: chat (UltraChat), math (NuminaMath-CoT), code (CodeFeedback) and multilingual (Aya, non-English). About 6k tokens per domain per model gives 34–36 conversations. Each is formatted with the model’s own chat template and nothing else, to avoid shared-template contamination [23]. Prompt tokens are prefill; only assistant tokens are decode steps. The conversations are replayed back to back in a fixed shuffled order, as one user’s session, so cache state carries across conversations and domains.

Cost. Everything ran on a 2-vCPU, 7 GB cloud VM with no GPU. Qwen3-30B-A3B (61 GB of bf16 weights) took about 30 minutes, and gpt-oss-20b about 25 minutes, mostly spent reading weights over the network. We also traced gpt-oss-120b but exclude it: its teacher-forced perplexity on the chat corpus is implausibly high, although its weights match an independent bf16 checkpoint exactly and it gives a normal perplexity on plain text. Until the cause is understood, we do not use its routing.

4 A Bytes-over-Bandwidth Decode Model

Model. For one decode step,

$$T = \frac{D + X_g}{\eta_g B_g} + \frac{X_c}{\eta_c B_c} + n_c \tau_e + \frac{X_f}{\eta_p B_p} + n_f \tau_x + n_h \tau_h. \quad (1)$$

D is the per-token dense traffic: attention, shared experts, dense MLPs, routers, norms, the LM head and KV-cache reads. X_g , X_c and X_f are routed-expert bytes executed from GPU memory, executed on the CPU (n_c expert executions), and fetched over PCIe (n_f transfers). B_g , B_c and B_p are datasheet peak bandwidths of GPU memory, host DRAM (channels $\times$ MT/s $\times$ 8 B) and the host–device

link. The τ s are fixed latencies per CPU expert, per transfer and per GPU $\leftrightarrow$ CPU hand-off. Byte counts are exact: each architecture is instantiated on PyTorch’s `meta` device and every parameter classified. Bits per weight come from the quantisation format, or from the model’s file size when the source reports it.

Validation data. We collected published decode measurements that state the hardware. Each row cites its source and quotes the table or line; there are 52 points from 9 independent sources. They cover:

- CPU-only execution on 2- to 12-channel DDR5;
- static CPU offload in `llama.cpp`, `ik_llama.cpp`, `KTransformers` and a 2026 research engine [18];
- all-in-VRAM baselines;
- demand fetch over PCIe [4].

The 8 models are gpt-oss-20b and 120b, Qwen3-30B-A3B and 235B-A22B, DeepSeek-V3/R1, Kimi-K2, and Mixtral 8 $\times$ 7B and 8 $\times$ 22B. Four rows with unstated DRAM speed (assumed DDR5-6000), approximate web-UI readings and VM runs are flagged in the data. Two-socket runs of `llama.cpp`-family engines are held out before fitting (see below).

Fitting and held-out evaluation. We fit η and τ by least squares on $\log(\hat{y}/y)$ and evaluate by *leave-one-source-out* cross-validation: all rows from one paper or thread are predicted by a model fitted on the rest. The demand-fetch rows come from a single paper, so they are held out one GPU at a time, giving 12 held-out groups in total. Table 1 shows the ablation.

- Bytes over bandwidth plus a hand-off term (M0) already reaches 21% median error.
- A per-transfer latency (M1) fixes the fetch rows, because the naive whole-layer copy and the per-expert copy differ in transfer count, not in bytes.
- A per-CPU-expert latency (M4) fixes the small-expert models on the CPU, bringing the Qwen3-30B-A3B rows to 4%.
- Per-layer fixed costs (M2) and a server/desktop split of η_c (M3) do not help.

We report M4, the variant with the lowest held-out MAPE. Because it was picked after seeing every variant’s held-out error, its reported error is slightly optimistic; the gap to M1 is one point.

The fitted values are $\eta_g = 0.38$, $\eta_c = 0.60$, $\eta_p = 0.86$, $\tau_e = 21\mu\text{s}$ and $\tau_x = 5.8\text{ms}$. τ_h is driven to zero and absorbed by τ_e . The fitted τ_x describes one Python prototype, whose bulk copies run at $\approx 87\%$ of PCIe peak while each per-expert copy costs milliseconds. For all bounds and projections we therefore use $\tau_x = 10\mu\text{s}$, a DMA-engine latency.

Table 1: Model-term ablation. Median absolute percentage error and MAPE, leave-one-source-out.

Variant	params	median	MAPE
M0 bytes/BW + hand-off	4	21%	24%
M1 + per-transfer latency	5	17%	18%
M2 + per-layer fixed costs	7	17%	19%
M3 + server/desktop η_c	8	19%	20%
M4 M1 + CPU per-expert lat.	6	16%	17%

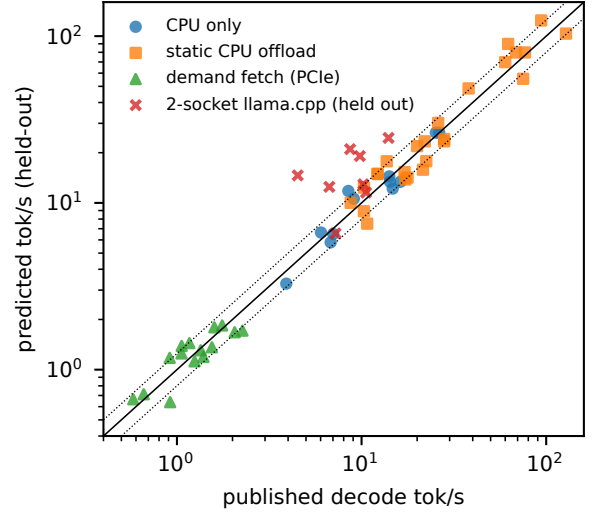


Figure 1: Held-out predictions vs. published decode throughput (log-log; dotted lines mark $\pm 25\%$). Crosses are two-socket `llama.cpp`-family runs, which were not used for fitting.

Results. The held-out median error is 16% (MAPE 17%), with 79% of points within 25% and the worst at 44% (Fig. 1). By execution style, the median error is 12% for CPU-only, 20% for static offload and 15% for demand fetch. Fitting a CPU efficiency per engine ranks the implementations as expected: 0.98 for the 2026 AVX-512 engine [18], 0.72 for `ik_llama.cpp`, 0.57 for `llama.cpp` and 0.45 for early `KTransformers` releases. The 8 held-out two-socket `llama.cpp`-family runs are over-predicted by a median of 81%. The effect is largest for DeepSeek, whose experts are small, and small for Mixtral 8 $\times$ 22B, whose experts are large. That points to cross-socket synchronisation per expert rather than bandwidth; the model has no term for it. The GPU-side efficiency η_g is the least constrained parameter: only three all-in-VRAM rows pin it, and they disagree by vendor. We therefore test every projection below with $\eta_g = 0.6$ as well.

A consistency check for dynamic caches. One public benchmark thread [13] runs DeepSeek-V4-Flash on the same two-GPU machine with a `llama.cpp` fork that keeps a GPU expert cache and runs misses on the CPU. It reports hit rates of 0%, 53%, 76% and 81% at 13.61, 15.84, 17.58

and 17.66 tok/s. Our serial-cache model makes per-token time affine in the hit rate: CPU time scales with misses, and the extra GPU work for hits is small. Calibrated on the 0% and 53% points, it predicts 17.05 and 17.32 tok/s for the other two, errors of 3% and 2%. The fitted CPU share of time is only 27%, which explains this rig’s modest gains. This validates the structure of the dynamic-cache model, not its absolute scale; no public measurement of a dynamic cache states all the inputs the model needs.

5 A Speed-of-Light Bound

Class. Consider any policy that keeps at most c of each layer’s experts resident on the GPU at any time. It may be static or dynamic, demand-driven or prefetching. Every routed-expert execution runs either on the GPU from a resident copy (cost $a = s/\eta_g B_g$ for an expert of s bytes) or on the CPU (cost $b = s/\eta_c B_c + \tau_e$). Each resident copy needs one load over PCIe (cost $p = s/\eta_p B_p$), and loads may overlap with anything.

Loads bound misses. Group each expert’s requests into maximal runs that are served by one resident copy. A run of r requests needs one load and *bridges* $r-1$ reuse gaps while occupying a slot. Belady’s MIN with bypass maximises the number of bridged gaps under the per-layer capacity limit; its hits are exactly the bridged gaps. We verify its optimality against exhaustive search on random instances. A prefetching schedule bridges no more gaps than some demand schedule with bypass, because shortening each resident interval to start at the first request it serves keeps every bridged gap and never increases occupancy. Experts resident at the start count as loads. Hence, for any policy in the class, GPU executions $\leq G^* + \text{loads}$, and so

$$\text{CPU executions} \geq M^* - \text{loads}, \quad (2)$$

where G^* and M^* are MIN-bypass’s hits and misses. A prefetcher can turn misses into hits, but each conversion costs a load.

Bound. Let x be the mean number of loads per layer step and m_c the mean number of CPU executions. Each layer step takes at least $\max((k-m_c)a, m_cb)$ if the CPU and GPU work concurrently, or $(k-m_c)a + m_cb$ if they serialise. Both are convex in m_c , so by Jensen’s inequality the mean over layer steps is at least the value at the mean $m_c \geq \max(0, M^*/N - x)$. Across the run, the link must also carry the loads, so time per token is at least Lxp . Minimising over $x \geq 0$ gives the bound

$$T \geq \min_{x \geq 0} \max \left(T_D + L \min_{m_c \geq (M^*/N - x)^+} \ell(m_c), Lxp \right), \quad (3)$$

with ℓ the concurrent or serial layer cost and T_D the dense time. Hand-offs, per-transfer latency and the capacity cost of prefetching are all dropped, so the bound remains valid under the model. A regression test checks it on six

platforms and three parameter sets against LRU, LFU, MIN and an optimistic oracle prefetcher that loads every MIN-bypass admission one step early; the tightest policy reaches 98% of the bound. The static layer split and a global cross-layer pool fall outside the class, because both give some layers more than c slots. We still report them for comparison. With concurrent CPU–GPU execution, the bound can exceed all-in-VRAM speed, since the CPU adds bandwidth.

When is copying an expert worth it? A copy costs p once. Each later GPU execution then saves $b-a$ compared with the CPU. The copy therefore pays off only if the expert is reused at least

$$r^* = \frac{p}{b-a} = \frac{1/(\eta_p B_p)}{1/(\eta_c B_c) - 1/(\eta_g B_g)} \quad (4)$$

times while resident. The CPU cost b includes the fitted per-expert latency τ_e , so r^* depends on expert size. For Qwen3-30B it is 1.6 on an RTX 4090 (PCIe 4, DDR5-6000) and 0.8 on an RTX 5090 (PCIe 5, DDR5-6400). It is 4.4 on an RTX 4060 (PCIe 4 $\times 8$, DDR5-5600), where the narrow link makes copies expensive and GPU execution is only about twice as fast as the CPU. Values below one mean that fetching a miss outright, at cost $p+a$, is cheaper than running it on the CPU, at cost b . On the PCIe 4 platforms $r^* > 1$ for all three models; on PCIe 5 it can fall below one.

6 Results

6.1 Routing locality

Table 2 and Fig. 2 summarise routing behaviour.

Short-range reuse is strong in every model. For Qwen3-30B, 46% of selections repeat an expert chosen by the *previous* token, $7.4\times$ the uniform-routing rate, and 88% repeat one chosen within the last 16 tokens. A 25% LRU cache therefore hits 82%, against 91% for the offline optimum. LRU still incurs $2.1\times$ as many misses.

Long-range popularity is weak, and whether it transfers across domains depends on the model. The most popular quarter of experts receives only about half of all selections. In Qwen3, the popular sets of different domains overlap *less* than random sets would (Jaccard 0.11 vs. 0.14). Pinning the experts that are hottest on the other three domains then hits only 14% on the held-out domain, below the 25% of a static layer split with the same memory. In OLMoE the overlap is somewhat above random (0.19), and held-out pinning (32%) does beat the layer split. In both models, pinning calibrated on the evaluation data itself (“oracle”) flatters static placement considerably.

6.2 Throughput against the bound

Figure 3 and Table 3 turn hit rates into throughput with the validated model. Rows for dynamic caches are projec-

Table 2: Routing statistics over decode tokens, averaged over layers. Reuse@ w is the fraction of selections whose expert was also selected within the previous w tokens (uniform routing in parentheses). Top-25% share is the fraction of selections that go to the most popular quarter of experts. Domain Jaccard is the overlap between different domains’ top-25% sets; random sets give 0.14. Hit rates are for a GPU cache holding 25% of each layer’s experts. Brackets are 95% bootstrap intervals over conversations.

	OLMoE-1B-7B	Qwen3-30B-A3B	gpt-oss-20b
experts E / active k / MoE layers	64/8/16	128/8/48	32/4/24
decode tokens; teacher-forced ppl	20,115; 3.67	20,480; 5.43	18,595; 17.49
reuse@1 (uniform)	46% [43–50] (12.5)	46% [43–49] (6.2)	50% [47–53] (12.5)
reuse@16	90%	88%	92%
top-25% share	49%	49%	47%
domain Jaccard	0.19	0.11	0.14
hit: static layer split	25%	25%	25%
hit: hot experts, held-out domain	32% [28–37]	14% [13–16]	30% [25–34]
hit: hot experts, oracle	49%	49%	47%
hit: LRU	65% [61–69]	82% [80–84]	71% [68–73]
hit: Belady (MIN)	77%	91%	81%
hit: MIN-bypass	80%	91%	84%

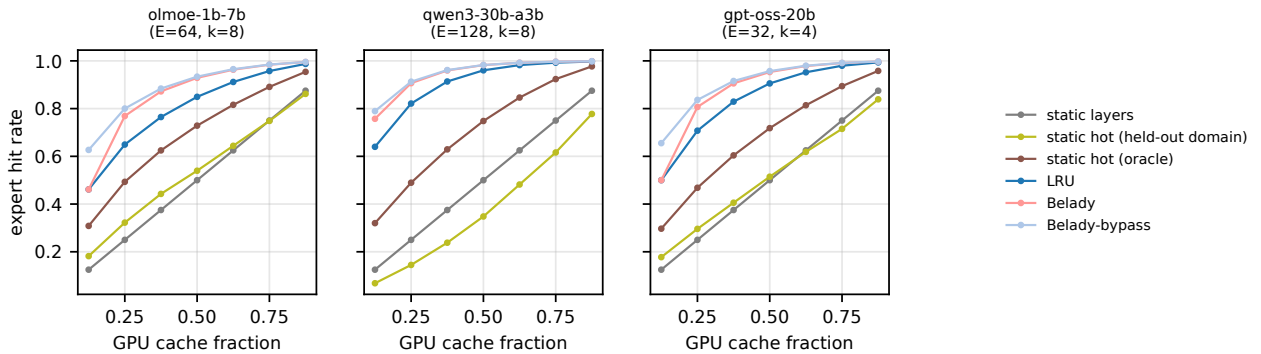


Figure 2: Expert hit rate against GPU cache size (as a fraction of each layer’s experts) for static and dynamic policies.

tions: they rest on the structural check in Section 4, not on a direct measurement.

(i) **Where misses execute matters more than the replacement policy.** On both PCIe 4 platforms, the same LRU cache with CPU-executed misses is never slower than with fetch-on-miss (worst-case ratio 1.01), for every model, cache size and hand-off latency we evaluate (0–50 μ s). This is consistent with desktop users settling on llama.cpp’s CPU offload rather than on fetch-based research systems. On PCIe 5 the order is no longer fixed: fetching is up to 14% faster in some configurations, as $r^* < 1$ predicts.

(ii) **On an 8 GB GPU, a simple dynamic cache captures most of the bound.** This is the setting where these models actually need offloading. There, llama.cpp’s static layer split reaches 67% (gpt-oss-20b) and 64% (Qwen3) of the bound. A per-layer LRU cache with CPU-executed misses reaches 86–91%, a 1.28–1.42 $\times$ speed-up. The cache is given nearly the same memory; it holds 1–2 percentage points more of the experts because whole layers round down. If CPU and GPU work serialise, the

speed-up is smaller (Table 3). With $\eta_g = 0.6$ it grows to 1.46–1.65 $\times$, and LRU then reaches 83–86% of the bound.

(iii) **On faster GPUs with small budgets, the static split wastes most of the attainable speed.** With a quarter of the experts resident on the RTX 4090 and 5090, across the three models:

- the static split reaches 16–28% of the bound;
- LRU with CPU misses reaches 32–65%, or 1.6–3.0 $\times$ the static split;
- an optimistic oracle prefetcher, which is one attainable point and not a cap, reaches 70–91%.

The gap between LRU and the bound is the most that any prefetcher or learned predictor can add under per-layer capacity. It is small on the 8 GB GPU, where LRU already matches the oracle, and substantial on fast GPUs.

(iv) **On the RTX 4060, all strategies gain less.** Copying is unattractive there ($r^* = 4.4$ for Qwen3), and GPU expert throughput is only about twice the CPU’s. Every strategy therefore stays within 1.59 $\times$ of all-experts-on-CPU speed, and DRAM bandwidth is the main lever.

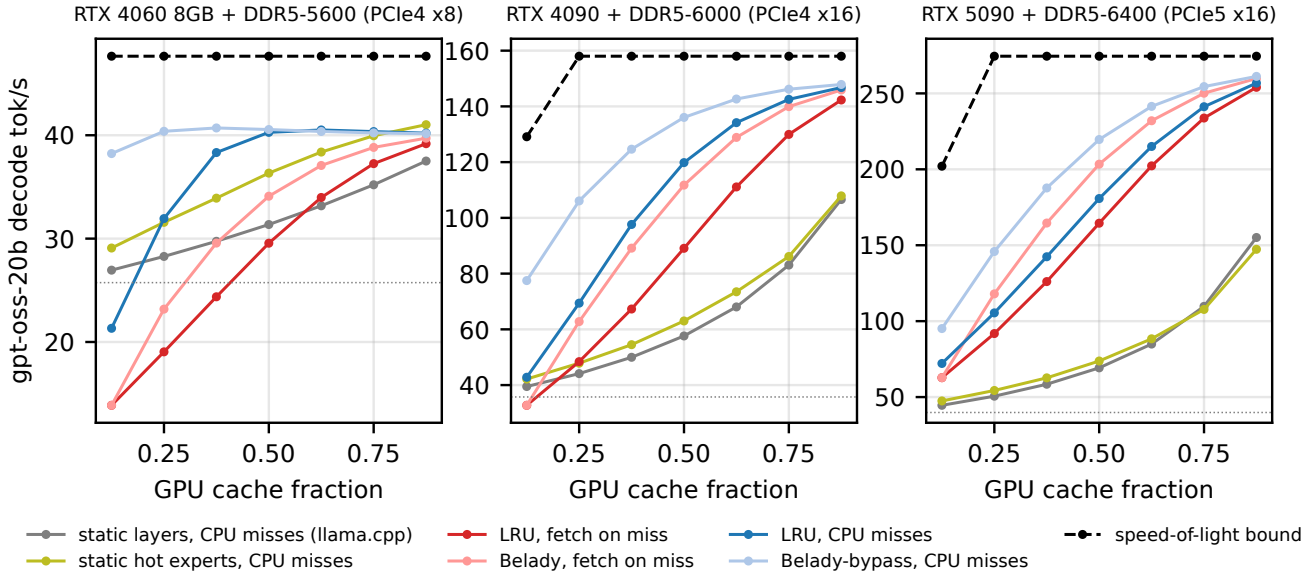


Figure 3: gpt-oss-20b decode tok/s against GPU cache size on three desktop platforms, using the validated model with a 20 μ s hand-off. Dashed: speed-of-light bound for per-layer policies. Dotted grey: all experts on the CPU.

7 Discussion

For engine builders. The first-order win on desktops is a *dynamic* expert cache with CPU-executed misses and asynchronous admission rather than better static placement. Prediction and prefetching are second-order on 8 GB-class GPUs but matter on fast GPUs with small budgets (Table 3). Admission should respect r^* : on PCIe 4, an expert should be copied only if it is likely to be reused a few times. We found no dynamic GPU expert cache in mainline llama.cpp. Community forks that add one report gains consistent with the affine model of Section 4, but the same thread also reports a slowdown on an older GPU (GTX 1080 Ti) [13]. That is the regime where r^* says copying rarely pays.

For evaluation. Hit rate is a poor proxy for speed once misses can run elsewhere. Policies should be reported as a fraction of a hardware-specific bound, with hand-off and admission costs stated. The released measurement set and model make that cheap.

For model design. Equation (1) and the traces also run in reverse. Given a target platform (B_g, B_c, B_p and VRAM), they predict decode speed as a function of expert count, expert size, top- k and shared-expert size, with locality measured rather than assumed. That suggests designing MoE architectures *for* a consumer memory hierarchy. We leave this co-design, which requires training models, to future work.

8 Limitations

- **Off-policy text.** Traces are teacher-forced on dataset responses, not on each model’s own samples. Routing is exact for the text, but self-generated text may be more predictable.
- **Batch 1 only.** Small-batch serving takes the union of experts across requests, which weakens locality.
- **Unmodelled platform effects.** The model has no NUMA or cross-socket term, no core-count limit, and no DRAM contention between the CPU expert path and admission copies. Fitted efficiencies absorb implementation quality, and η_g is weakly identified.
- **Retrodictive validation only.** The author had no hardware, so every measurement is third-party. Some are approximate or come from VMs, and four assume unstated DDR5 speeds. Dynamic-cache projections are validated in structure only.
- **Scope of the bound.** It covers policies with per-layer capacity; a global pool across layers could do better.
- **gpt-oss-120b excluded.** Its trace has an unexplained perplexity anomaly (Section 3), so it is not used for any result.
- **Numerics.** Traces were computed in fp32 on the CPU. GPU inference in bf16 can break near-ties differently; re-tracing OLMoE with bf16 compute changes 6.6% of (token, layer) expert sets and moves the 25%-cache LRU hit rate by 0.0 points.

Table 3: Projected decode tok/s (validated model, fitted parameters). “8GB” columns are VRAM-matched: the experts that fit on an RTX 4060 after dense weights, an 8k-token KV cache and a 1.5 GB reserve (49% and 27% of experts). Both models fit entirely on the RTX 4090 and 5090, so those columns are what-ifs with a budget of 25% of the experts, standing in for larger models on the same GPUs. Percentages are relative to the concurrent speed-of-light bound. The last row repeats the LRU/static speed-up with $\eta_g = 0.6$.

Strategy	gpt-oss-20b			Qwen3-30B-A3B		
	4060 8GB	4090, 25%	5090, 25%	4060 8GB	4090, 25%	5090, 25%
all experts on CPU	25.0	36	41	21.2	34	38
static layer split (llama.cpp)	29.5 (67%)	45 (28%)	52 (19%)	23.3 (64%)	42 (26%)	48 (17%)
LRU, fetch on miss	27.0	48	92	23.3	73	132
LRU, CPU misses, serial	35.4	68	101	30.3	96	130
LRU, CPU misses, concurrent	37.8 (86%)	70 (44%)	109 (40%)	33.1 (91%)	107 (65%)	143 (50%)
MIN-bypass, CPU misses	38.1 (86%)	108 (69%)	151 (55%)	33.6 (92%)	134 (81%)	197 (68%)
oracle prefetch, CPU misses	37.8	124 (78%)	202 (73%)	33.4	150 (91%)	252 (87%)
speed of light (serial / concurrent)	37.6 / 44.1	143 / 158	263 / 274	33.2 / 36.4	159 / 165	282 / 289
LRU-concurrent / static	1.28×	1.6×	2.1×	1.42×	2.5×	3.0×
with $\eta_g = 0.6$ (8GB)	1.46×			1.65×		

9 Conclusion

A causal model’s routing is fixed by its prefix, so exact decode traces of large MoEs can be collected on a small CPU machine without storing a single checkpoint. A six-parameter bytes-over-bandwidth model, held out source by source across 9 independent sources, predicts batch-1 decode to within about a sixth. Combined with Belady, load accounting and Jensen, these give a speed-of-light bound for consumer MoE inference. The bound shows that on today’s desktops, where a miss executes matters first, and how cleverly it is predicted matters second, mostly on fast GPUs.

Artifact. The code, traces and annotated measurement set are released, together with scripts that regenerate every number and figure. The repository also includes a 30-minute protocol for adding a first-party measurement.

Acknowledgments. Parts of the code, the literature search and the text were drafted with the help of an AI system (Claude, Anthropic). An independent AI review pass found errors in an earlier version of the bound, and these were corrected. The author reviewed the work and takes responsibility for its content.

References

- [1] Laszlo A. Belady. A study of replacement algorithms for a virtual-storage computer. *IBM Systems Journal*, 5(2):78–101, 1966.
- [2] Pei Cao, Edward W. Felten, Anna R. Karlin, and Kai Li. A study of integrated prefetching and caching strategies. In *Proceedings of ACM SIGMETRICS*, 1995.
- [3] Shiyi Cao, Shu Liu, et al. MoE-Lightning: High-throughput MoE inference on memory-constrained GPUs. *arXiv preprint arXiv:2411.11217*, 2024.
- [4] Artyom Eliseev and Denis Mazur. Fast inference of mixture-of-experts language models with offloading. *arXiv preprint arXiv:2312.17238*, 2023.
- [5] Georgi Gerganov and contributors. llama.cpp: LLM inference in C/C++. <https://github.com/ggml-org/llama.cpp>, 2023–2026.
- [6] Torsten Hoefer, Dan Alistarh, Tal Ben-Nun, Nikoli Dryden, and Alexandra Peste. Sparsity in deep learning: Pruning and growth for efficient inference and training in neural networks. *Journal of Machine Learning Research*, 22(241), 2021.
- [7] Torsten Hoefer and Roberto Belli. Scientific benchmarking of parallel computing systems: Twelve ways to tell the masses when reporting performance results. In *Proceedings of the International Conference for High Performance Computing, Networking, Storage and Analysis (SC)*, 2015.
- [8] En-Ming Huang, Li-Shang Lin, et al. Efficient CPU-GPU collaborative inference for MoE-based LLMs on memory-limited systems. *arXiv preprint arXiv:2512.16473*, 2025.
- [9] Ranggi Hwang, Jianyu Wei, et al. Pre-gated MoE: An algorithm-system co-design for fast and scalable mixture-of-expert inference. *arXiv preprint arXiv:2308.12066*, 2023.
- [10] Andrei Ivanov, Nikoli Dryden, Tal Ben-Nun, Shigang Li, and Torsten Hoefer. Data movement is all you need: A case study on optimizing transformers. In *Proceedings of Machine Learning and Systems (MLSys)*, 2021.
- [11] Keisuke Kamahori, Tian Tang, et al. Fiddler: CPU-GPU orchestration for fast inference of mixture-of-experts models. *arXiv preprint arXiv:2402.07033*, 2024.
- [12] KVCache.AI and contributors. KTransformers: A flexible framework for CPU/GPU hybrid LLM inference. <https://github.com/kvcache-ai/ktransformers>, 2024–2026. See also the SOSP 2025 paper by the same team.
- [13] llama.cpp contributors. llama.cpp discussion #24528: MoE expert cache benchmarks. <https://github.com/ggml-org/llama.cpp/discussions/24528>, 2026. Accessed September 2026.

- [14] Xinyu Qiu, Chuhong Xu, et al. GGUF-metadata prediction of single-sequence llama.cpp throughput across three systems. *arXiv preprint arXiv:2609.14864*, 2026.
- [15] Yuan Si, Yufeng Lin, et al. Who should own the expert cache? kernel-managed tiering for trillion-parameter MoE inference. *arXiv preprint arXiv:2608.12103*, 2026.
- [16] Xiaoni Song, Zihang Zhong, et al. ProMoE: Fast MoE-based LLM serving using proactive caching. *arXiv preprint arXiv:2410.22134*, 2024.
- [17] Peng Tang, Jiacheng Liu, et al. HOBBIT: A mixed precision expert offloading system for fast MoE inference. *arXiv preprint arXiv:2411.01433*, 2024.
- [18] Wenxin Wang, Yule Hou, et al. Achieving cloud-grade SLOs for local mixture-of-experts inference through CPU-GPU hybrid design. *arXiv preprint arXiv:2606.10493*, 2026.
- [19] Zihan Wang, Yuqi Wang, et al. SeqMoE: Toward full-load performance via predictive and graph-compatible MoE offloading. *arXiv preprint arXiv:2609.12978*, 2026.
- [20] Samuel Williams, Andrew Waterman, and David Patterson. Roofline: An insightful visual performance model for multicore architectures. *Communications of the ACM*, 52(4):65–76, 2009.
- [21] Leyang Xue, Yao Fu, et al. MoE-Infinity: Efficient MoE inference on personal machines with sparsity-aware expert cache. *arXiv preprint arXiv:2401.14361*, 2024.
- [22] Yichao Yuan, Lin Ma, et al. MoE-Lens: Towards the hardware limit of high-throughput MoE LLM serving under resource constraints. *arXiv preprint arXiv:2504.09345*, 2025.
- [23] Yu Zhang. Reproducible evaluation of MoE expert caching: Replay semantics, workload contamination, and operating regimes. *arXiv preprint arXiv:2608.07911*, 2026.
- [24] Yushu Zhao, Yubin Qin, et al. MoBiLE: Efficient mixture-of-experts inference on consumer GPU with mixture of big little experts. *arXiv preprint arXiv:2510.12357*, 2025.
- [25] Shuzhang Zhong et al. HybriMoE: Hybrid CPU-GPU scheduling and cache management for efficient MoE inference. In *Design Automation Conference (DAC)*, 2025. arXiv:2504.05897.
- [26] Shien Zhu, Samuel Bohl, et al. Pre-attention expert prediction and prefetching for mixture-of-experts large language models. *arXiv preprint arXiv:2511.10676*, 2025.